



Versión del Documento 4.0

Equipo 4

- *Carpinteiro Tapia Mariam Ximena*
- *Espinosa Roque Rebeca*
- *García Díaz Michelle Ariadna*
- *Guillén Gilabert Lara*
- *Nava Córdova Mariana*
- *Sánchez Cortés Wendy*

Versión del Documento: 4.0

Fecha de actualización: 25 de mayo de 2026

Historial de cambios:

Versión	Cambios
1.0	Versión inicial
2.0	Integración frontend, autenticación, perfil de usuario
3.0	Nuevos CU: registrar animal, publicar, buscar, ver ficha informativa
4.0	Nuevos CU: manifestar interés por adoptar un animal, cambiar contraseña, borrar publicación, integración con las APIs de razas, visualizar animal en mapa, actualizar animal

Backend: Versión 4.0.0 (Tag publicado en el repositorio remoto).

- 5 nuevos casos de uso CU10, CU11, CU12, CU13, CU14

Frontend: Versión 3.0.0 (Tag publicado en el repositorio remoto).

- Búsqueda de animales con filtros (especie, tamaño, código postal, raza)
- Feed de publicaciones con carrusel de fotos
- Botón "Me interesa" en tarjetas de animales

- Formulario de publicación de animal
- Mapa con ubicaciones relativas

1. Introducción

1.1 Propósito del documento

1.2 Alcance del Sistema

1.3 Definiciones relevantes

2. Actores del Sistema

3. Requerimientos Funcionales

4. Casos de uso

CU1 - Registro de nuevo usuario

CU2 — Iniciar sesión

CU4 — Cerrar sesión

CU5 — Actualizar datos

CU6 — Publicar animal en adopción

CU8 — Buscar animales publicados para adopción

CU9 — Manifestar interés en adopción

CU10 — Editar publicación

CU11 — Ver ficha informativa de un animal

CU12 — Permitir al usuario subir fotos

CU13 — Eliminar publicación

CU14 — Visualizar animales en mapa

CU15 — Restablecer contraseña olvidada

CU16 — Consultar animales de interés (Favoritos)

5. Requerimientos No Funcionales

6. Tecnologías Utilizadas

6.1 Lenguaje de programación

6.2 Framework.

6.3 Base de datos

6.4 Herramientas relevantes utilizadas en el desarrollo.

6.5 Tecnologías del Frontend

6.6 Manejo de Imágenes

6.7 Información de las razas

7. Arquitectura general

Modelo MVC

Arquitectura REST

[MVC adaptado a REST](#)

[Arquitectura en Capas del Sistema](#)

[Integración Frontend — Backend](#)

[Integración de APIs TheDogAPI y TheCatAPI](#)

[8. Flujo de Autenticación](#)

[9. Modelo Conceptual de Datos](#)

[Entidad Usuario](#)

[Atributos](#)

[Entidad Animal](#)

[Entidad Raza \(Breed\)](#)

[Entidad Foto \(Photo\)](#)

[Entidad Favorito \(Animal Favorite\)](#)

[Consideraciones de seguridad](#)

[10. Limitaciones actuales](#)

1. Introducción

1.1 Propósito del documento

Este documento tiene como propósito describir el estado actual del sistema desarrollado hasta la Iteración 4 del proyecto. Su objetivo es proporcionar una visión clara y estructurada de las funcionalidades implementadas, así como de los requerimientos funcionales y no funcionales que actualmente cumple el sistema.

Este documento funciona como una referencia para facilitar la comprensión de la organización del sistema, y como un medio de comunicación para stakeholders, permitiendo entender qué hace el sistema en su estado actual. Asimismo, sirve como base para futuras iteraciones, en las cuales se extenderán las funcionalidades del sistema de acuerdo con los objetivos del proyecto.

1.2 Alcance del Sistema

El sistema implementa funcionalidades de gestión de usuarios y de gestión de animales en adopción dentro de la plataforma, integradas en un flujo completo desde la interfaz de usuario hasta la persistencia en base de datos.

Actualmente, el sistema permite:

- El registro de usuarios mediante la captura de información básica.
- La autenticación de usuarios a través de credenciales (correo electrónico y contraseña).
- El cierre de sesión, invalidando la sesión activa del usuario.
- La consulta de la información del usuario autenticado mediante un endpoint protegido.
- La actualización del username, nombre, apellido y código postal.
- El restablecimiento de la contraseña mediante un enlace temporal enviado por correo.
- El registro y publicación de animales en adopción, con carga de fotografías.
- La búsqueda de animales mediante filtros (especie, tamaño y código postal).
- La consulta de la ficha informativa de cada animal, incluyendo información de raza obtenida de una API externa.
- La edición y eliminación de las publicaciones propias.
- La manifestación de interés en un animal, notificando por correo a su dueño y registrándolo en los favoritos del usuario.
- La consulta de los animales marcados como de interés (favoritos).
- La visualización de los animales en un mapa interactivo mediante su ubicación aproximada por código postal.

El sistema prioriza aspectos básicos de seguridad, como el almacenamiento cifrado de contraseñas y la validación de credenciales.

1.3 Definiciones relevantes

- **Usuario registrado:** Persona que ha creado una cuenta proporcionando su nombre, correo electrónico y código postal, y que tiene acceso a las funcionalidades de publicación y contacto.
- **Adopción responsable:** Proceso facilitado por el sistema mediante el cual una persona asume el compromiso ético y legal de brindar cuidado, protección y bienestar a un animal.

- **Animal en adopción:** Perro o gato registrado en la plataforma por un usuario, disponible para ser adoptado.
- **Ubicación aproximada:** Referencia geográfica basada únicamente en el *código postal*, utilizada para mostrar al animal en un mapa sin revelar la dirección exacta del domicilio.
- **Ficha informativa del animal:** Sección informativa dentro del sistema que muestra los datos detallados del animal, incluyendo nombre, descripción, tipo, raza, fotografías y ubicación aproximada.
- **Interés de adopción:** Funcionalidad que permite a un usuario manifestar interés en adoptar un animal, generando una notificación automática por correo electrónico al usuario que lo registro.
- **Autenticación:** Proceso mediante el cual el sistema verifica la identidad de un usuario mediante credenciales.
- **Contraseña cifrada:** Contraseña almacenada mediante un algoritmo de cifrado o hash de manera que no pueda leerse en texto plano.
- **Dato de contacto sensible:** Información personal que el sistema protege y no muestra directamente en la interfaz pública para evitar spam o riesgos de seguridad.
- **API externa:** Servicio externo que permite obtener información adicional.

2. Actores del Sistema

- **Usuario registrado**

Persona que tiene una cuenta en el sistema y puede acceder a funcionalidades protegidas como consulta de su información y cierre de sesión.

- **Usuario visitante**

Persona que no cuenta con una sesión activa en el sistema. Puede registrarse o iniciar sesión.

- **Soporte técnico**

Personal encargado del mantenimiento del sistema. No interactúa directamente con las funcionalidades implementadas en esta iteración.

3. Requerimientos Funcionales

RF1: Registro de usuarios

El sistema permite el registro de usuarios capturando el nombre completo, una dirección de correo electrónico válida, el código postal de residencia y una contraseña para el acceso a la cuenta.

RF2: Autenticación de usuarios

El sistema autentifica a los usuarios registrados mediante su correo electrónico y contraseña antes de permitir el acceso a las funcionalidades de la plataforma.

RF3: Consulta de usuario autenticado

El sistema permite obtener la información de usuario autenticado mediante un endpoint protegido (`/me`).

RF4: Cierre de sesión

El sistema permite al usuario registrado cerrar su sesión activa, invalidando el token de autenticación.

RF5: Actualización de datos del perfil de usuario

El sistema permite al usuario autenticado actualizar su username, nombre, apellido y código postal.

RF6: Registro de animal en adopción

El sistema permite registrar perros o gatos con nombre, especie, tamaño, descripción, código postal y fotos.

RF7: Publicación de animal

El sistema permite crear publicaciones visibles en el feed de otros usuarios.

RF8: Búsqueda de animales

El sistema permite filtrar animales por especie, tamaño y código postal.

RF9: Manifestar interés

El sistema permite a un usuario marcar interés en un animal, notificando por correo al dueño con los datos del interesado.

RF10: Recuperación de contraseña

El sistema permite a un usuario solicitar el restablecimiento de su contraseña mediante su correo electrónico, recibiendo un enlace temporal de un solo uso para definir una nueva contraseña.

RF11: Visualización de animales de interés

El sistema permite al usuario autenticado consultar la lista de animales en los que ha manifestado interés (favoritos).

4. Casos de uso

flowchart TB

actorV((Usuario Visitante))

actorR((Usuario Registrado))

subgraph Sistema

UC1[Registro de usuario]

UC2[Iniciar sesión]

UC3[Consultar usuario autenticado (/me)]

UC4[Cerrar sesión]

UC5[Actualizar datos]

UC6[Registrar animal en adopción]

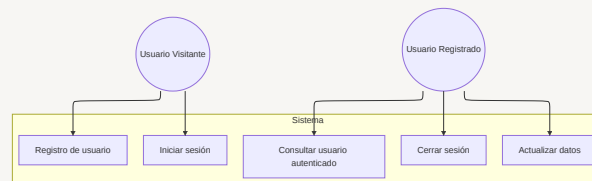
end

actorV --> UC1

actorV --> UC2

actorR --> UC3

actorR --> UC4
actorR --> UC5
actorR --> UC6



CU1 - Registro de nuevo usuario

Actores:	Usuario visitante (no registrado)
Descripción breve:	Permite a una persona crear una cuenta en el sistema para poder acceder a las funcionalidades de la plataforma.

Precondición: El usuario no tiene una sesión activa en la plataforma.

Flujo principal:

1. El usuario accede a la opción "Crear cuenta".
2. El usuario ingresa su nombre completo, correo electrónico y código postal.
3. El usuario establece una contraseña para su cuenta.
4. El sistema valida que el correo tenga un formato valido.
5. El sistema cifra la contraseña antes de guardarla.
6. El Sistema confirma la creación de la cuenta y redirige al inicio de sesión.

Flujos alternativos:

A1: Si el correo ya existe en la base datos, el sistema muestra un error y solicita uno diferente.

A2: Si el formato de correo es inválido, el sistema muestra un error y solicita un correo valido.

Poscondición: La cuenta del usuario queda habilitada en el sistema.

CU2 — Iniciar sesión

Actores	Usuario registrado, Servicio de autenticación
Descripción	El usuario accede al sistema mediante sus credenciales para utilizar las funcionalidades de la plataforma.

Precondición: El usuario tiene una cuenta registrada y no tiene sesión activa.

Flujo principal:

1. El usuario accede a la opción "Iniciar sesión".
2. El usuario ingresa su correo electrónico y contraseña.
3. El sistema envía las credenciales al servicio de autenticación.
4. El servicio valida las credenciales y emite un token de sesión.
5. El el sistema devuelve una respuesta con el token de autenticación.

Flujos alternativos:

A1: Si las credenciales son incorrectas, el sistema muestra un mensaje de error sin especificar cuál campo es incorrecto.

A2: Si el usuario falla 5 intentos consecutivos, el sistema bloquea temporalmente el acceso.

Postcondición: El usuario accede a la plataforma con una sesión autenticada activa.

CU4 — Cerrar sesión

Actores	Usuario registrado, Servicio de autenticación
Descripción	El usuario finaliza su sesión activa en la plataforma para proteger su cuenta y evitar accesos no autorizados en el dispositivo actual.

Precondición: El usuario debe tener una sesión autenticada activa en el sistema.

Flujo principal:

1. El usuario selecciona la opción "Cerrar sesión".
2. El sistema solicita al Servicio de autenticación la invalidación del token de sesión actual.
3. El Servicio de autenticación confirma la revocación del token o su expiración inmediata.
4. El sistema elimina las cookies o datos de sesión almacenados localmente en el navegador/aplicación.
5. El sistema redirige al usuario a la página de inicio o al formulario de Inicio de Sesión.

Flujos alternativos:

- **A1: Error de conexión con el servicio de identidad:** Si el servicio externo no responde, el sistema debe forzar de todas formas el borrado de la sesión local para impedir que el usuario siga navegando, aunque el token no se haya invalidado en el servidor inmediatamente.
- **A2: Cierre por inactividad:** Si el usuario no interactúa con el sistema por un tiempo prolongado, el sistema ejecuta el cierre de sesión de forma automática.

Postcondición: El token de sesión queda invalidado en la base de datos y el usuario vuelve al estado de "Usuario no autenticado", perdiendo acceso a las funcionalidades privadas.

CU5 — Actualizar datos

Actores	Usuario registrado
Descripción	Permite al usuario autenticado actualizar sus datos personales (nombre, apellido, username y código postal.)

Precondición: El usuario debe tener una sesión autenticada activa en el sistema.

Flujo principal:

1. El usuario accede a la vista de "Actualización de perfil".
2. El sistema muestra un formulario con los datos actuales del usuario .
3. El usuario modifica uno o más campos y guarda los cambios.
4. El sistema envía la solicitud HTTP al backend mediante el método PUT al endpoint /users, incluyendo el token de autenticación.
5. El backend valida la validez del token y procesa la actualización en la base de datos.
6. El sistema confirma la actualización exitosa y actualiza la información de la vista.

Flujos alternativos:

- **A1: Token inválido o ausente.**

CU6 — Publicar animal en adopción

Actores:	Usuario registrado
Descripción breve:	El usuario publica un animal en adopción en la plataforma.

Precondición: El usuario ha iniciado sesión exitosamente.

Flujo principal:

1. El usuario selecciona la opción " New post ".

2. El sistema muestra el formulario de registro.
3. El usuario ingresa el nombre del animal, la especie y el código postal (campos obligatorios). Opcionalmente puede ingresar fecha de nacimiento, descripción y tamaño.
4. Una vez seleccionada la especie, el sistema consulta la API externa de razas y despliega las opciones disponibles. El usuario puede seleccionar una raza o dejar el campo vacío.
5. El usuario adjunta entre 1 y 5 fotografías del animal en formato JPG, PNG o WEBP. El sistema muestra una previsualización de cada imagen seleccionada.
6. El usuario envía el formulario.
7. El sistema valida que los campos obligatorios estén completos y que se haya adjuntado al menos una fotografía.
8. El sistema sube cada fotografía a Cloudinary y obtiene una URL pública por imagen.
9. El sistema registra el animal en la base de datos junto con las URLs de las fotografías.
10. El sistema confirma el registro con el mensaje *"Animal registered successfully!"* y cierra el formulario.

Flujos alternativos:

A1: Si falta algún campo obligatorio (nombre, especie o código postal), el sistema muestra el mensaje *"Name, species and zipcode are required."* y no envía el formulario.

A2: Si el usuario no adjunta ninguna fotografía, el sistema muestra el mensaje *"At least one photo is required."* y no envía el formulario.

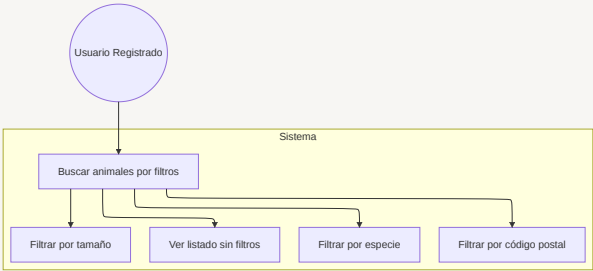
A3: Si el usuario intenta seleccionar más de 5 fotografías, el sistema descarta las sobrantes y muestra el mensaje *"You can only upload up to 5 photos."*

A4: Si falla la carga de una o más fotografías en Cloudinary, el sistema muestra el mensaje *"Failed to upload one or more photos."* y no continúa con el registro.

Poscondición: El post queda publicado y visible para otros usuarios en la plataforma.

CU8 — Buscar animales publicados para adopción

```
flowchart TB
    actorR((Usuario Registrado))
    subgraph Sistema
        UC1[Buscar animales por filtros]
        UC2[Ver listado sin filtros]
        UC3[Filtrar por especie]
        UC4[Filtrar por tamaño]
        UC5[Filtrar por código postal]
    end
    actorR --> UC1
    UC1 --> UC2
    UC1 --> UC3
    UC1 --> UC4
    UC1 --> UC5
```



Actores:	Usuario registrado (autenticado)
Descripción breve:	Permite a un usuario autenticado consultar el listado de animales disponibles para adopción, con la posibilidad de aplicar filtros opcionales para refinar los resultados.

Precondición: El usuario tiene una sesión activa en la plataforma (token Bearer válido).

Flujo principal:

1. El usuario accede a la sección de animales en adopción.
2. El sistema realiza una petición `GET /animals` con el token de autenticación.
3. El sistema retorna todos los animales registrados, ordenados del más reciente al más antiguo.
4. El usuario visualiza el listado con la información de cada animal (especie, tamaño, código postal, raza y fotos).

Flujos alternativos:

A1 — Filtro por especie: El usuario selecciona una especie (`DOG` o `CAT`). El sistema retorna únicamente los animales que coincidan con dicha especie.

A2 — Filtro por tamaño: El usuario selecciona un tamaño (`small` , `medium` , `large` o `extra_large`). El sistema retorna únicamente los animales que coincidan con ese tamaño.

A3 — Filtro por código postal: El usuario ingresa un código postal (5–10 caracteres). El sistema retorna únicamente los animales cuyo código postal coincida de forma exacta.

A4 — Combinación de filtros: El usuario aplica dos o más filtros simultáneamente. El sistema aplica todos los criterios de forma conjunta y retorna solo los animales que cumplan todas las condiciones.

A5 — Sin resultados: Si ningún animal coincide con los filtros aplicados, el sistema retorna un arreglo vacío `[]` con estado `200 OK` . No se genera un error `404` .

Restricciones (fuera de alcance en esta versión):

- No es posible seleccionar múltiples valores para un mismo filtro (ej. `DOG` y `CAT` simultáneamente).
- No existe búsqueda por nombre, raza o descripción libre del animal.
- No existe filtro por edad o rango de fechas.
- No existe búsqueda por proximidad geográfica; el código postal es una coincidencia exacta.

Poscondición: El usuario visualiza el listado de animales disponibles, filtrado según los criterios seleccionados.

CU9 — Manifestar interés en adopción

Actores	Usuario registrado
Descripción	El usuario expresa interés en adoptar un animal publicado en la plataforma. El sistema registra el interés y notifica por correo electrónico al dueño del animal con los datos del interesado.

Precondición: El usuario tiene una sesión activa en la plataforma.

Flujo principal:

1. El usuario visualiza la ficha de un animal en la sección de búsqueda.
2. El usuario presiona el botón "Me interesa".
3. El sistema registra el interés en la base de datos asociando al usuario con el animal.
4. El sistema envía un correo electrónico al dueño del animal con el nombre y correo del interesado, así como la ficha del animal.
5. El botón cambia a "¡Interés enviado! 🐾" y se deshabilita para evitar duplicados.

Flujos alternativos:

A1: Si el usuario ya había manifestado interés en ese animal previamente, el sistema rechaza la solicitud y el interés no se registra de nuevo.

Poscondición: El interés queda registrado en la base de datos como favorito del usuario (consultable desde "My Favorites"), y el dueño del animal recibe una notificación por correo electrónico.

CU10 — Editar publicación

Actores:	Usuario registrado y un animal en adopción
Descripción breve:	El usuario modifica el nombre, especie, fecha de nacimiento, tamaño, código postal y descripción de un animal publicado previamente.

Precondición: El usuario ha iniciado sesión exitosamente y es el dueño de la publicación a editar.

Flujo principal:

1. El usuario selecciona la opción de editar en la tarjeta de un animal de su propiedad.
2. El sistema muestra un formulario con los datos actuales del animal: nombre, especie, fecha de nacimiento, tamaño, código postal y descripción.
3. El usuario modifica uno o más campos y confirma los cambios.
4. El sistema envía una solicitud `PUT /animals/{id}` con el token de autenticación.
5. El backend valida que el token sea válido y que el usuario autenticado sea el propietario del animal.
6. El backend actualiza los datos en la base de datos.
7. El sistema confirma la actualización y cierra el formulario.

Flujos alternativos:

A1: Si el usuario no es el propietario del animal vinculado, el sistema rechaza la operación y muestra un mensaje de error.

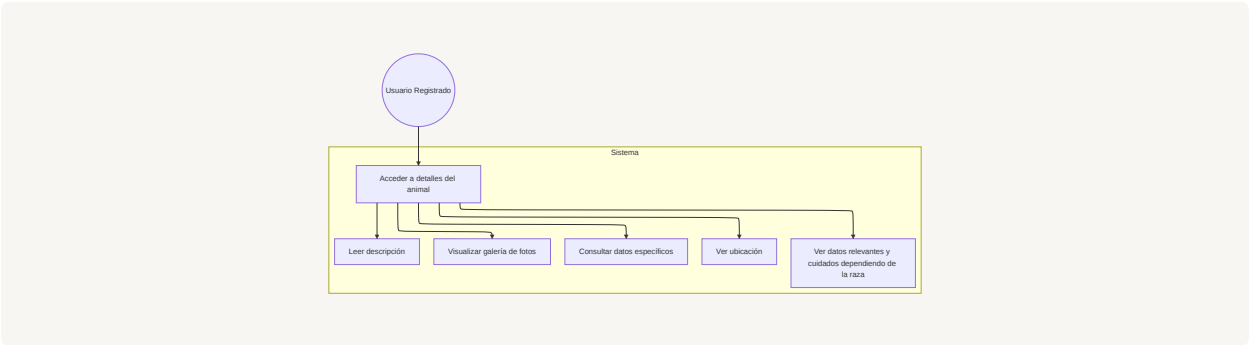
A2: Si no se modifica ningún campo, el sistema mantiene la información existente sin cambios.

Restricciones:

- No es posible modificar las fotografías del animal desde esta vista.
- No es posible modificar la raza del animal una vez registrado.

Poscondición: La publicación queda actualizada con la nueva información proporcionada por el usuario.

CU11 — Ver ficha informativa de un animal



Actores:	Usuario registrado con autenticación
Descripción breve:	El usuario puede visualizar una versión completa de los detalles sobre una mascota específica.

Precondición: El usuario tiene una sesión activa (token de sesión válido) y tiene una mascota en mente de la que quiere saber más información ubicada en el `PostFeed` o encontrada mediante `AnimalSearch`.

Flujo principal:

1. El usuario hace clic en el botón "More info" de la tarjeta de un animal.
2. El sistema redirige a la ruta `/animal/:id` y realiza una petición `GET /animals/{id}` con el token de autenticación.
3. El sistema devuelve el objeto con los detalles del animal.
 - a. Hace primero consultas a la base de dato para la totalidad de los datos a excepción del mapa e información de cuidados específicos y datos sobre la raza.
 - b. Para los datos de la raza y cuidados primero revisa la tabla de razas local para ver si ya se tiene un registro anterior de la raza que se desea. De lo contrario pregunta la información a la API correspondiente y guarda la información en la base de datos.
4. El sistema renderiza la ficha informativa.
5. El usuario visualiza el nombre, raza, cuidados, especie, la galería de fotos, tamaño, descripción y ubicación en el mapa.

Flujos alternativos:

- **A1 — Navegación de fotos:** El usuario interactúa con los controles del carrusel de fotos para visualizar las distintas fotos del animal.
- **A2 — Animal no encontrado:** Si el `id` en la URL no existe o fue eliminado, el sistema captura el error 404 a través del `ExceptionHandler` y muestra el mensaje "Animal no encontrado".

Restricciones:

- La información es de solo lectura, excepto el mapa con la ubicación del animal, el cual es interactivo.

Poscondición: El usuario queda informado sobre las características del animal para decidir si le interesa adoptar o no al animal.

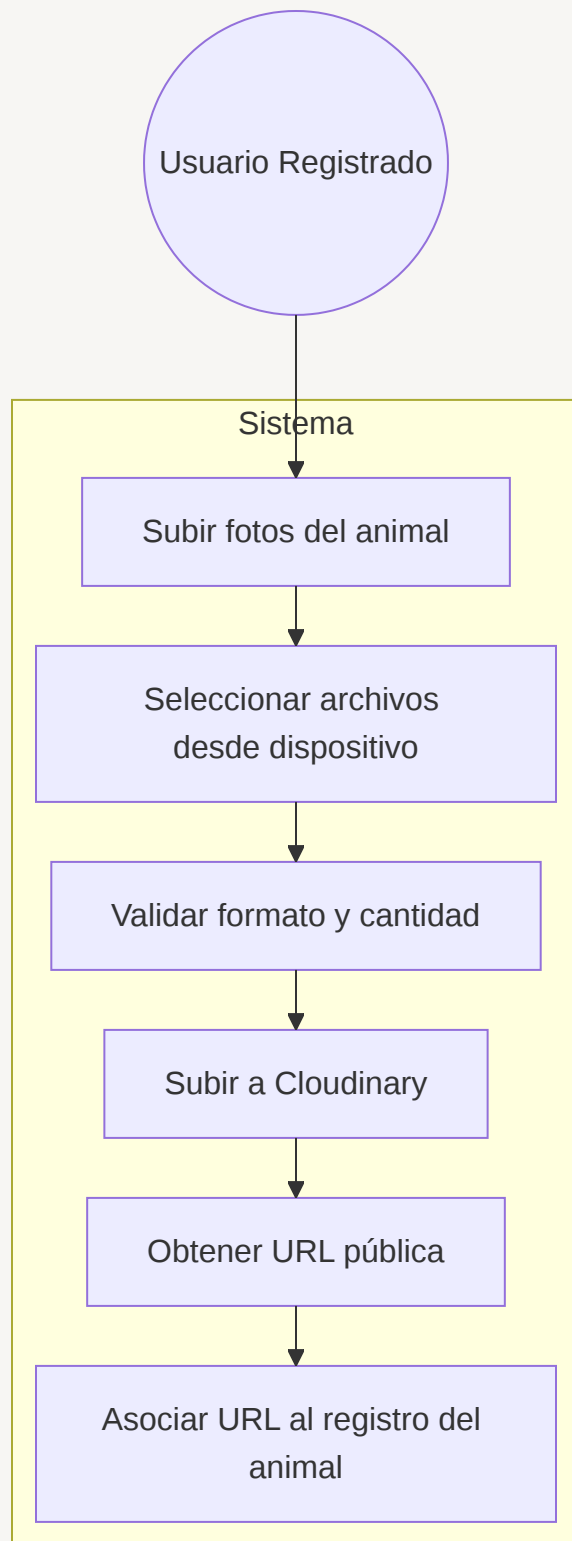
CU12 — Permitir al usuario subir fotos

flowchart TB

```

actorR((Usuario Registrado))
subgraph Sistema
    UC1[Subir fotos del animal]
    UC2[Seleccionar archivos desde dispositivo]
    UC3[Validar formato y cantidad]
    UC4[Subir a Cloudinary]
    UC5[Obtener URL pública]
    UC6[Asociar URL al registro del animal]
end
actorR --> UC1
UC1 --> UC2
UC2 --> UC3
UC3 --> UC4
UC4 --> UC5
UC5 --> UC6

```



Actores:	Usuario registrado (autenticado)
Descripción breve:	Permite a un usuario autenticado seleccionar y subir entre 1 y 5 fotografías de un animal directamente desde su dispositivo. Las imágenes se almacenan en Cloudinary y sus URLs públicas se asocian al registro del animal en el sistema.

Precondición: El usuario tiene una sesión activa en la plataforma (token Bearer válido) y se encuentra en el formulario de registro de animal.

Flujo principal:

1. El usuario accede al formulario de publicación de animal.
2. El usuario hace clic en el área de carga de fotos.
3. El sistema abre el selector de archivos del dispositivo.
4. El usuario selecciona entre 1 y 5 imágenes en formato JPG, PNG o WEBP.
5. El sistema muestra una previsualización de las imágenes seleccionadas.
6. El usuario completa los demás campos del formulario y presiona **Post**.
7. El sistema sube cada imagen directamente a Cloudinary mediante `POST` `<https://api.cloudinary.com/v1_1/{cloud}/image/upload>`.
8. Cloudinary retorna una URL pública (`secure_url`) por cada imagen.
9. El sistema registra el animal enviando las URLs al backend mediante `POST` `/animals/register`.
10. El backend guarda las URLs en la tabla `photo` asociadas al animal.
11. El sistema muestra el mensaje *"Animal registered successfully!"* y cierra el formulario.

Flujos alternativos:

- **A1 — Cantidad excedida:** El usuario intenta seleccionar más de 5 fotos. El sistema muestra el mensaje *"You can only upload up to 5 photos"* y descarta las fotos sobrantes.
- **A2 — Eliminar foto antes de enviar:** El usuario presiona el botón `×` sobre una previsualización. El sistema elimina esa foto de la selección sin afectar las

demás.

- **A3 — Error al subir a Cloudinary:** Si una o más imágenes no pueden subirse, el sistema muestra *"Failed to upload one or more photos"* y no continúa con el registro del animal.
- **A4 — Error al registrar el animal:** Si el backend rechaza el registro (campos inválidos, restricciones de BD), el sistema muestra el mensaje de error correspondiente y conserva las fotos ya subidas a Cloudinary.

Restricciones (fuera de alcance en esta versión):

- Solo se admiten formatos JPG, PNG y WEBP.
- El límite es de 5 fotos por animal.
- No es posible reordenar las fotos una vez seleccionadas.
- No existe edición ni recorte de imágenes dentro de la plataforma.
- Las imágenes subidas a Cloudinary no se eliminan automáticamente si el registro del animal falla.

Poscondición: El animal queda registrado en el sistema con sus URLs de fotos almacenadas en la tabla `photo`, apuntando a imágenes alojadas en Cloudinary accesibles públicamente desde cualquier dispositivo.

CU13 — Eliminar publicación

Actores:	Usuario registrado (autenticado) y una publicación de animal en adopción
Descripción breve:	El usuario elimina una publicación de animal que él mismo registró, confirmando la acción desde la vista de edición.

flowchart TB

actorR((Usuario Registrado))

subgraph Sistema

UC1[Seleccionar publicación a editar]

UC2[Mostrar información actual de la publicación]

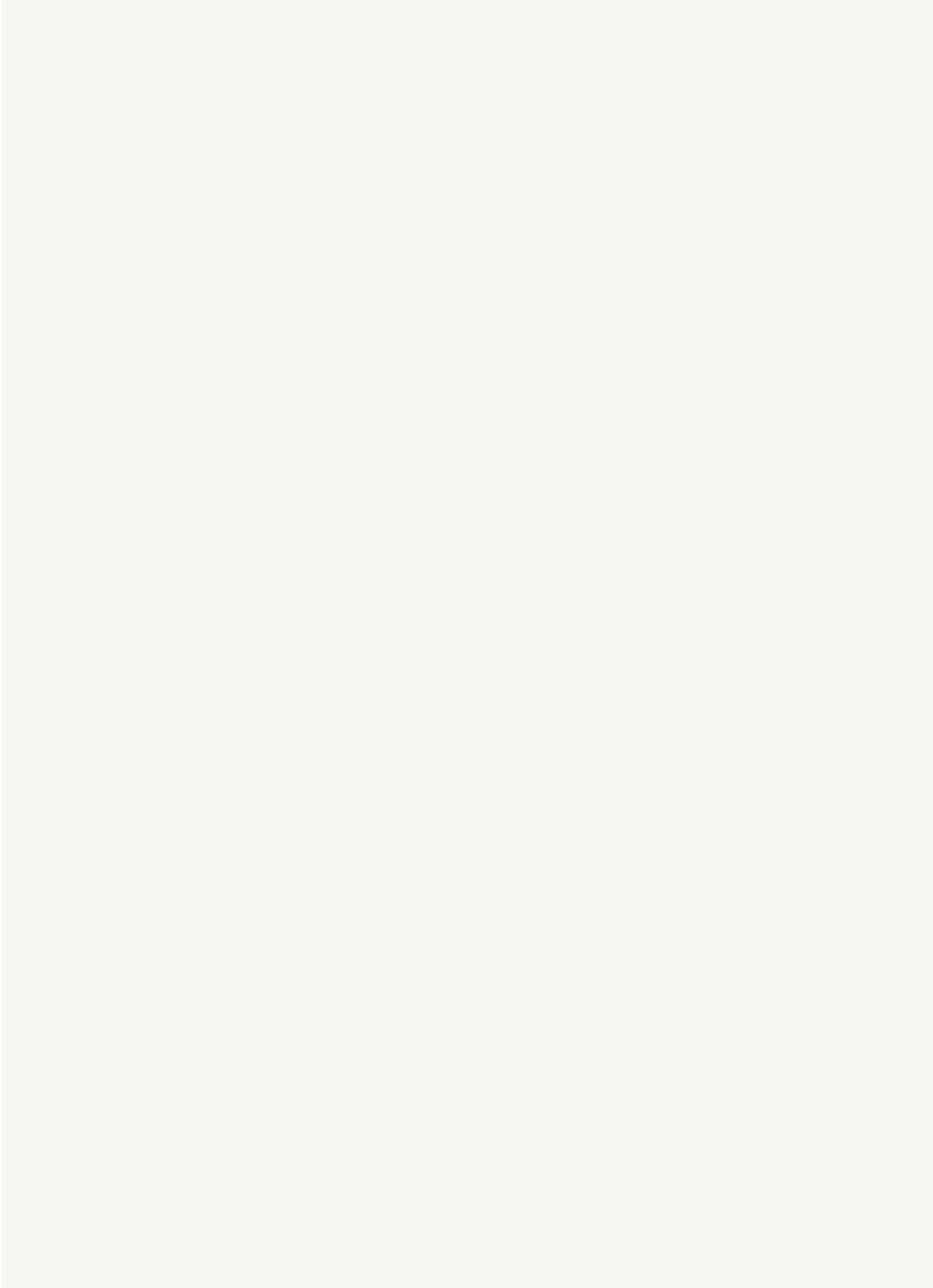
UC3[Seleccionar opción Delete]

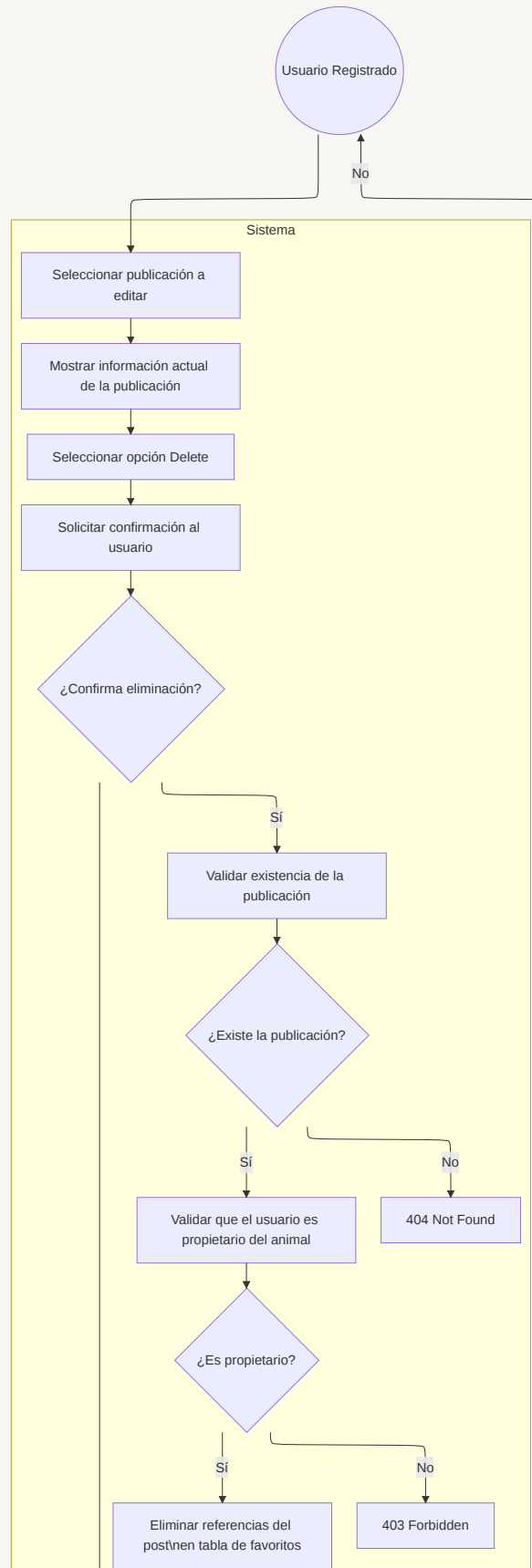
UC4[Solicitar confirmación al usuario]

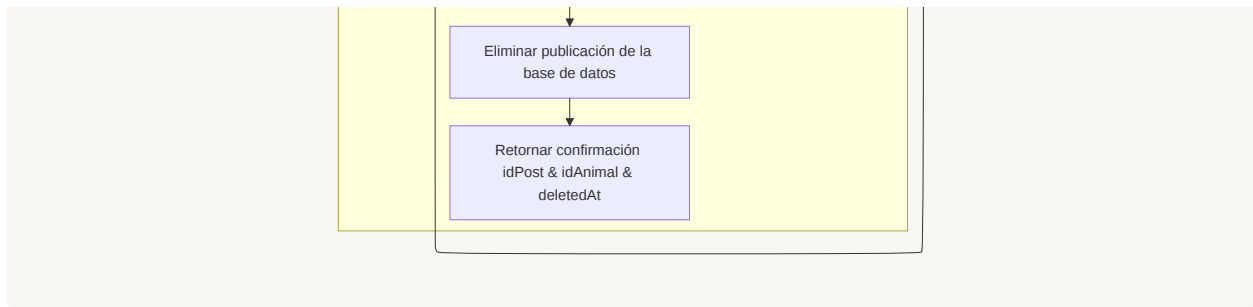
```

UC5{¿Confirma eliminación?}
UC6[Validar existencia de la publicación]
UC7{¿Existe la publicación?}
UC8[Validar que el usuario es propietario del animal]
UC9{¿Es propietario?}
UC10[Eliminar referencias del post\nen tabla de favor
itos]
UC11[Eliminar publicación de la base de datos]
UC12[Retornar confirmación idPost & idAnimal & delete
dAt]
ERR1[404 Not Found]
ERR2[403 Forbidden]
end
actorR --> UC1
UC1 --> UC2
UC2 --> UC3
UC3 --> UC4
UC4 --> UC5
UC5 -- No --> actorR
UC5 -- Sí --> UC6
UC6 --> UC7
UC7 -- No --> ERR1
UC7 -- Sí --> UC8
UC8 --> UC9
UC9 -- No --> ERR2
UC9 -- Sí --> UC10
UC10 --> UC11
UC11 --> UC12

```







Precondición: El usuario ha iniciado sesión exitosamente y es el dueño del animal vinculado a la publicación.

Flujo principal:

1. El usuario selecciona la publicación que desea editar.
2. El sistema muestra la información actual de la publicación junto con la opción de eliminarla.
3. El usuario selecciona la opción de eliminar.
4. El sistema solicita confirmación al usuario.
5. El usuario confirma la eliminación.
6. El sistema valida que el usuario sea el propietario del animal vinculado a la publicación.
7. El sistema elimina la publicación y retorna la confirmación con el ID del post, el ID del animal y la fecha de eliminación.

Flujos alternativos:

A1: Si la publicación no existe, el sistema retorna un error 404 y detiene la operación.

A2: Si el usuario no es el propietario del animal vinculado, el sistema rechaza la operación con un error 403 y muestra un mensaje de error.

A3: Si el usuario no confirma la eliminación, el sistema cancela la operación y mantiene la publicación sin cambios.

A4: Si ocurre un error interno durante la eliminación, el sistema retorna un error 500.

Poscondición: La publicación queda eliminada del sistema.

CU14 — Visualizar animales en mapa

```
flowchart TB
    actorU((Usuario Autenticado))
    subgraph Sistema
        UC1[Sollicitar coordenadas por CP]
        UC2[Validar Token y CP]
        UC3[Consultar API Geocoding - Mapbox]
        UC4[Renderizar Mapa Leaflet]
        UC5[Dibujar radio de aproximación]
        UC6[Posicionar marcadores de animales]
    end
    actorU --> UC1
    UC1 --> UC2
    UC2 --> UC3
    UC3 --> UC4
    UC4 --> UC5
    UC5 --> UC6
```



Sistema

Solicitar coordenadas por CP

Validar Token y CP

Consultar API Geocoding - Mapbox

Renderizar Mapa Leaflet

Dibujar radio de aproximación

Posicionar marcadores de animales

Actores:	Usuario registrado (autenticado)
Descripción breve:	Permite al usuario visualizar en un mapa interactivo la zona donde se encuentra un animal (o varios) basándose en su Código Postal. El sistema protege la privacidad del usuario mostrando un círculo de proximidad en lugar de una dirección exacta.

Precondición: El usuario tiene una sesión activa (Token Bearer) y el animal tiene un código postal (zipcode) válido registrado.

Flujo principal:

1. El usuario accede a la ficha de un animal (o a Home).
2. El sistema detecta el `zipcode` asociado al animal o animales.
3. El sistema solicita las coordenadas al backend mediante `GET /geocoding/{zipcode}` enviando el token en el header.
4. El backend valida el token y consulta el servicio de Geocoding (Mapbox).
5. El backend retorna un objeto JSON con `lat` y `lng`.
6. El sistema inicializa el mapa (Leaflet) centrado en dichas coordenadas.
7. El sistema dibuja un círculo de estilo personalizado (color `#63b7bf`, radio ~1200m) para representar el área aproximada.
8. **Caso Individual (Ficha):** Se muestra un solo mapa con el área del animal.
9. **Caso General (Home):** El sistema itera sobre la lista de animales y posiciona marcadores aleatorios dentro de los radios de sus respectivos CPs.

Flujos alternativos:

- **A1 — Código Postal no encontrado:** Si el servicio de geocoding no devuelve coordenadas (404), el sistema muestra el mensaje: *"Couldn't load a map for CP {zipcode}"*.
- **A2 — Error de autenticación:** Si el token es inválido o expiró (401/403), el mapa no se carga y se registra el error en consola.
- **A3 — Fallo de conexión:** Si la API de OpenStreetMap o el backend fallan, se muestra un estado de error visual en el contenedor del mapa.

Restricciones (fuera de alcance en esta versión):

- La ubicación es **aproximada** (centroide del CP).
- Requiere conexión a internet para cargar los mapas de OpenStreetMap.
- **Comportamiento del Buscador (Search Bar):** Al realizar una búsqueda por CP en el Home, el sistema filtra los resultados en la lista, pero el mapa no se recentra automáticamente de forma inmediata hacia la nueva ubicación.
- No se permite la edición de la ubicación desde el mapa.

Poscondición: El usuario visualiza la zona de los animales encontrados, manteniendo la privacidad del usuario exacta del rescatista.

CU15 — Restablecer contraseña olvidada

flowchart TB

actorR((Usuario Registrado))

subgraph Sistema

UC1[Solicitar enlace de recuperación]

UC2[Ingresar correo electrónico]

UC3[Generar token temporal - 1 hora]

UC4[Enviar correo con enlace]

UC5[Abrir enlace de restablecimiento]

UC6[Ingresar y confirmar nueva contraseña]

UC7[Validar token y vigencia]

UC8[Cifrar y guardar nueva contraseña]

ERR1[Enlace inválido o expirado]

end

actorR --> UC1

UC1 --> UC2

UC2 --> UC3

UC3 --> UC4

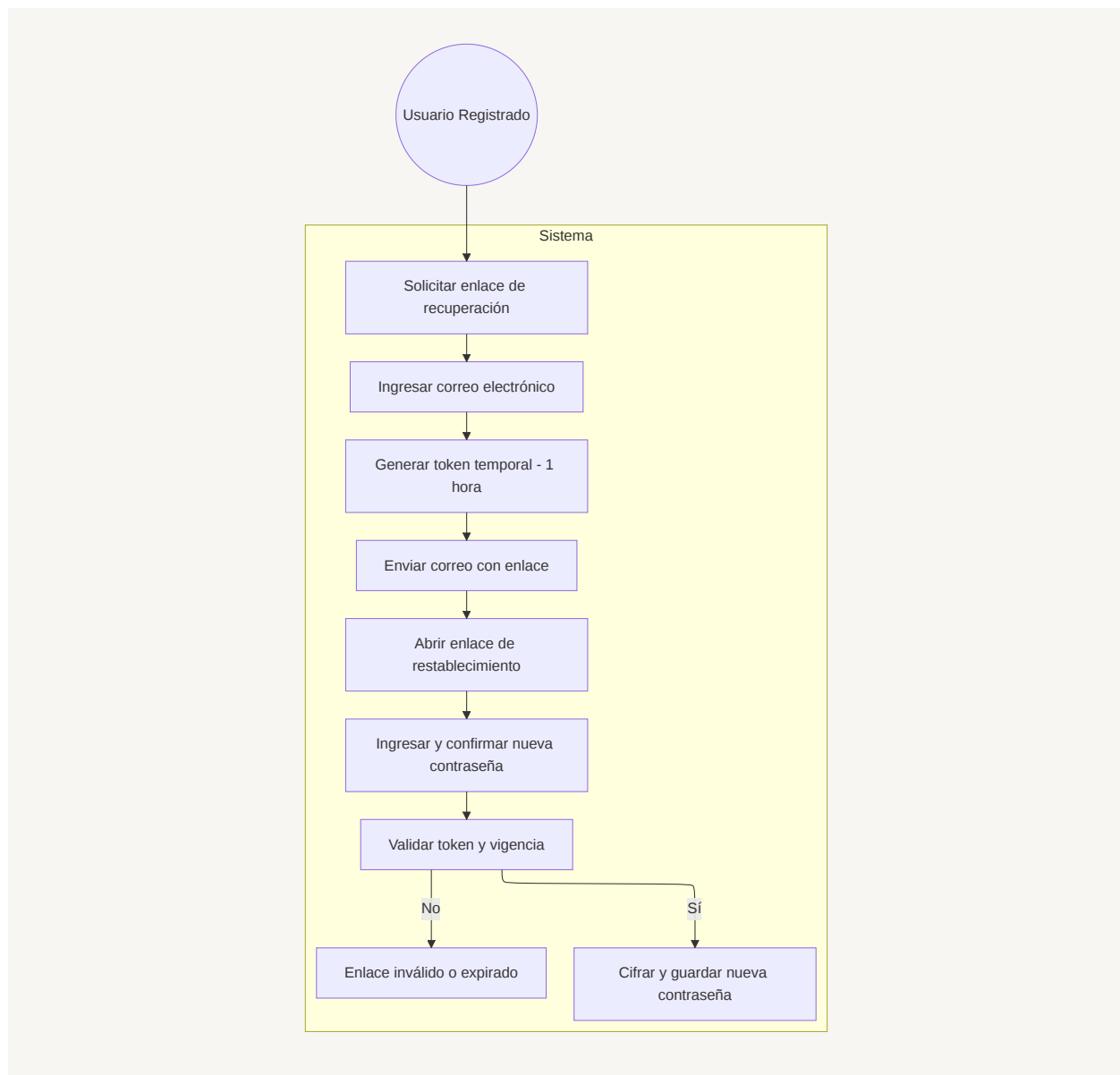
UC4 --> UC5

UC5 --> UC6

UC6 --> UC7

UC7 -- No --> ERR1

UC7 -- Sí --> UC8



Actores:	Usuario registrado
Descripción breve:	El usuario que olvidó su contraseña solicita un enlace de recuperación por correo y define una nueva contraseña.

Precondición: El usuario tiene una cuenta registrada y no recuerda su contraseña.

Flujo principal:

1. El usuario accede a la opción "Forgot your password?" desde la pantalla de inicio de sesión.
2. El usuario ingresa su correo electrónico y lo envía.

3. El sistema genera un token de restablecimiento único con una vigencia de 1 hora y lo asocia a la cuenta.
4. El sistema envía un correo electrónico al usuario con un enlace que contiene dicho token.
5. El usuario abre el enlace, que lo dirige a la vista de restablecimiento.
6. El usuario ingresa la nueva contraseña y la confirma.
7. El sistema valida el token, cifra la nueva contraseña, la guarda e invalida el token.
8. El sistema confirma el cambio y redirige al inicio de sesión.

Flujos alternativos:

A1: Si el correo no está registrado, el sistema responde igualmente con un mensaje genérico ("Si ese correo está registrado, se envió un enlace") para no revelar si la cuenta existe.

A2: Si el token es inválido o ya expiró (más de 1 hora), el sistema muestra el mensaje "This link is invalid or has expired" y no actualiza la contraseña.

A3: Si las dos contraseñas ingresadas no coinciden, el sistema muestra "Passwords do not match" y no envía la solicitud.

Poscondición: La contraseña del usuario queda actualizada y cifrada, y el token de restablecimiento queda invalidado.

CU16 — Consultar animales de interés (Favoritos)

flowchart TB

actorR((Usuario Registrado))

subgraph Sistema

UC1[Acceder a My Favorites]

UC2[Solicitar favoritos - GET /favorites/me]

UC3[Validar token de autenticación]

UC4[Obtener animales marcados como interés]

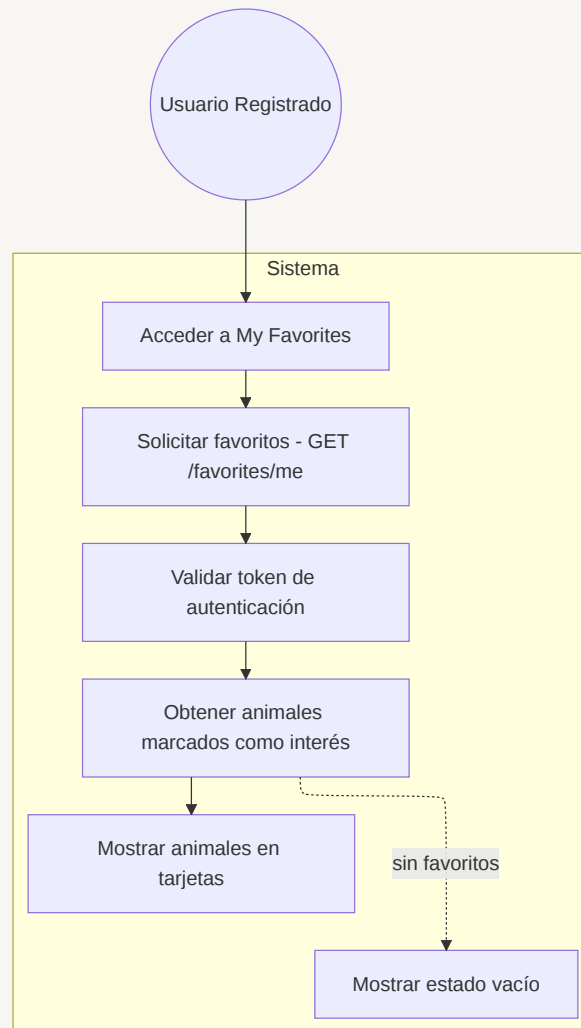
UC5[Mostrar animales en tarjetas]

UC6[Mostrar estado vacío]

```

end
actorR --> UC1
UC1 --> UC2
UC2 --> UC3
UC3 --> UC4
UC4 --> UC5
UC4 -->|sin favoritos| UC6

```



Actores:	Usuario registrado (autenticado)
Descripción breve:	El usuario consulta la lista de animales en los que ha manifestado interés mediante el botón "Me interesa".

Precondición: El usuario tiene una sesión activa y ha marcado interés en al menos un animal.

Flujo principal:

1. El usuario accede a la opción "My Favorites" desde el menú lateral.
2. El sistema realiza una petición `GET /favorites/me` con el token de autenticación.
3. El sistema retorna los animales que el usuario marcó como de interés.
4. El usuario visualiza dichos animales en el mismo formato de tarjetas del feed.

Flujos alternativos:

A1: Si el usuario no ha marcado interés en ningún animal, el sistema muestra un estado vacío indicando que aún no tiene favoritos.

Poscondición: El usuario visualiza el listado de animales en los que manifestó interés.

5. Requerimientos No Funcionales

RNF1 - Seguridad

Las contraseñas se almacenan mediante un algoritmo de cifrado seguro.

RNF2 – Rendimiento

Las solicitudes del sistema deben de responder en un tiempo promedio no mayor a 3 segundos bajo carga normal.

RNF3 - Disponibilidad

El sistema está disponible para los usuarios al menos el 99% del tiempo durante cada mes de operación.

RNF4 - Usabilidad

El sistema permite a un usuario registrarse e iniciar sesión de manera intuitiva sin necesidad de capacitación previa.

RNF6 - Privacidad

El sistema garantiza la protección de los datos personales de los usuarios, evitando la exposición pública de información sensible como direcciones exactas, contraseñas y correos electrónicos dentro de la interfaz de la plataforma. La información del usuario únicamente podrá ser utilizada para fines relacionados con la funcionalidad del sistema.

RNF6 - Compatibilidad

El sistema deberá ser compatible con los principales navegadores web modernos (Chrome, Firefox, Edge, etc) y ser accesible tanto desde dispositivos de escritorio como desde dispositivos móviles mediante diseño responsivo.

RNF7 - Escalabilidad

El sistema soporta múltiples usuarios concurrentes realizando operaciones de autenticación sin degradación perceptible del rendimiento.

RNF8 - Mantenibilidad

El código fuente del sistema se encuentra documentado y estructurado de forma modular, permitiendo que un desarrollador nuevo pueda identificar y corregir un error en un módulo sin afectar el funcionamiento del resto del sistema.

RNF11 - Integridad de Datos

El sistema garantiza la integridad de datos en la base de datos haciendo uso de políticas de mantenimiento adecuadas, no permitiendo la existencia de datos aislados o incompletos según la definición de la base de datos.

6. Tecnologías Utilizadas

6.1 Lenguaje de programación

| Kotlin

Kotlin es una opción acertada para el desarrollo de aplicaciones backend modernas por las grandes ventajas que ofrece en comparación a otros lenguajes del campo. Kotlin destaca por su sintaxis concisa y expresiva, lo que permite reducir significativamente la cantidad de código, facilitando así la lectura y el mantenimiento de código. Además, tiene un soporte para programación concurrente mediante *coroutines*, las cuales permiten escribir código asíncrono de manera más sencilla y legible, evitando la complejidad asociada con la gestión de múltiples hilos de ejecución. Otra de sus características importantes es su sistema de tipos con manejo explícito de valores nulos, este enfoque ayuda a prevenir errores asociados a referencias nulas.

6.2 Framework.

| Spring Boot

Spring Boot fue seleccionado como Framework por su compatibilidad con Java, y por ende, con Kotlin. Según IMB, Spring ofrece soporte integrado para tareas típicas que una aplicación necesita realizar, como enlace de datos, conversión de tipos, validación, manejo de excepciones, gestión de recursos y eventos, internacionalización y más. Spring permite la *inyección de dependencias* y proporciona configuraciones automáticas. Además, tiene una amplia comunidad, documentación extensa y soporte para tecnologías modernas.

6.3 Base de datos

| PostgreSQL

PostgreSQL fue elegido por su estabilidad, alto rendimiento y cumplimiento de los estándares SQL. Sigue la filosofía de código abierto y tiene una amplia comunidad. Tiene soporte para transacciones complejas, integridad de datos y

optimización de consultas. Todo esto es deseable para gestionar la información de nuestros usuarios y sus animales a adopción de forma segura y eficiente.

6.4 Herramientas relevantes utilizadas en el desarrollo.

| *Hibernate*

Hibernate se utilizó como herramienta de ORM, permitiendo la interacción entre el sistema desarrollaron en Kotlin y la base de datos de PostgreSQL mediante objetos en lugar de consultar SQL directas. Esto reduce significativamente la cantidad de código para acceder a la base datos.

| *Postman*

Es nuestra principal herramienta para probar y depurar la API REST, ya que permite enviar solicitudes HTTP a los distintos endpoints del sistema y analizar la respuesta obtenida, lo cual facilitan la verificación del correcto funcionamiento de la API.

| *Git*

Git es sin duda el mejor sistema de control de versiones para gestionar el desarrollo proyectos en equipo. Podemos registrar los cambios, mantener un historial de las modificaciones y sobre todo propiciar el trabajo colaborativo entre los miembros de nuestro equipo mediante el uso del repositorio compartido.

6.5 Tecnologías del Frontend

React: Biblioteca de JavaScript utilizada para construir la interfaz de usuario mediante componentes reutilizables (bloques independientes de código que representan partes visuales de la aplicación). Se eligió por su amplia adopción en la industria, su ecosistema maduro y la facilidad para gestionar el estado de la aplicación (los datos que cambian mientras el usuario interactúa con la pantalla).

Vite: Como su nombre lo indica, Vite es una herramienta de construcción (build tool) que acelera y hace más eficiente el desarrollo web, específicamente para levantar el servidor de desarrollo y compilar el proyecto. Como resultado nos ayuda a obtener aplicaciones más ligeras, tiempos de carga mínimos y un flujo de trabajo fluido.

React Router: Biblioteca utilizada para gestionar la navegación entre vistas dentro de la aplicación. Permite definir rutas (direcciones URL como /login /home /perfil) y controlar el acceso a páginas según si el usuario está autenticado o no.

Fetch API: Mecanismo nativo del navegador utilizado para realizar peticiones HTTP (solicitudes de datos entre el frontend y el backend) hacia el servidor. Se eligió por no requerir dependencias externas adicionales y ser suficiente para las necesidades actuales del sistema.

sessionStorage: Mecanismo de almacenamiento del navegador utilizado para guardar el token de sesión (una cadena de texto única que identifica al usuario autenticado) durante su visita. Se eligió por su naturaleza temporal: los datos se eliminan automáticamente al cerrar el navegador, lo que reduce riesgos de seguridad.

6.6 Manejo de Imágenes

| Cloudinary

Cloudinary fue integrado como servicio de almacenamiento y gestión de imágenes en la nube. Se optó por esta solución debido a que permite que las fotografías de los animales sean accesibles públicamente desde cualquier dispositivo y entorno, independientemente de la máquina donde se ejecute el sistema. A diferencia del almacenamiento local, Cloudinary garantiza la persistencia y disponibilidad de las imágenes sin requerir configuración adicional en el servidor. Su plan gratuito ofrece 25 GB de almacenamiento y 25 GB de transferencia mensual, lo cual resulta más que suficiente para el alcance de este proyecto. La integración se realizó directamente desde el frontend mediante uploads sin firma (unsigned uploads), donde el cliente sube la imagen directamente a Cloudinary, obtiene una URL pública (secure_url) y la envía al backend para su almacenamiento en la base de datos. Esto simplifica la

arquitectura al no requerir modificaciones en el backend para el manejo de archivos.

6.7 Información de las razas

| TheDogAPI y TheCatAPI

Para tener información completa sobre las razas de perros y gatos que sean publicados en la plataforma incorporamos la comunicación con las APIs REST públicas **TheDogAPI** y **TheCatAPI**. Estas herramientas actúan como un repositorio global, dinámico y normalizado de conocimiento veterinario sobre las razas. Al integrarlas con el sistema aseguramos que el sistema valide en tiempo real la existencia e información de cada raza registrada bajo los estándares internacionales.

7. Arquitectura general

El sistema está compuesto por dos componentes independientes que se comunican entre sí a través de HTTP (el protocolo de comunicación de la web): el frontend y el backend.

El backend del sistema se implementa siguiendo el patrón **MVC adaptado a una arquitectura REST, organizado mediante una arquitectura en capas**. Este enfoque permite separar responsabilidades dentro del sistema, facilitando la mantenibilidad, la escalabilidad y la claridad en la estructura del código.

Modelo MVC

El patrón **MVC (Model–View–Controller)** es un modelo de diseño de software que separa una aplicación en tres componentes principales con el objetivo de organizar el código y facilitar su mantenimiento.

- **Model (Modelo):** representa los datos del sistema y las reglas asociadas a estos. Se encarga de gestionar la información y su interacción con la base de datos.
- **View (Vista):** es la capa encargada de presentar la información al usuario.

- **Controller (Controlador)**: actúa como intermediario entre el modelo y la vista, recibiendo las solicitudes del usuario, procesándolas y devolviendo una respuesta adecuada.

Arquitectura REST

La arquitectura **REST (Representational State Transfer)** es un estilo arquitectónico utilizado para diseñar servicios web que permiten la comunicación entre sistemas a través del protocolo HTTP.

En un sistema REST, los recursos del sistema se exponen mediante **endpoints** accesibles mediante operaciones HTTP como:

- **GET** para obtener información
- **POST** para crear nuevos recursos
- **PUT/PATCH** para actualizar información
- **DELETE** para eliminar recursos

Las respuestas del servidor se envían en formatos estructurados **JSON**.

MVC adaptado a REST

En el contexto de un *backend* REST, el patrón MVC se adapta de manera que:

- Los **Controllers** reciben las solicitudes HTTP de los clientes.
- El **Model** representa las entidades del sistema y su persistencia.
- La **View** es reemplazada por **respuestas JSON** enviadas al cliente.

Arquitectura en Capas del Sistema

Además del patrón MVC, el sistema se organiza mediante una **arquitectura en capas**, lo que permite dividir la aplicación en componentes con responsabilidades específicas. En la implementación actual se utilizan tres capas principales:

- **Controller:**

Esta capa es responsable de manejar las solicitudes HTTP provenientes de los clientes. En esta capa se definen los *endpoints* de la API REST que permiten interactuar con el sistema.

Los *controllers* reciben las peticiones del cliente, validan la estructura básica de los datos recibidos y delegan la lógica de negocio a la capa de servicios. Una vez que el procesamiento ha sido realizado, los *controllers* construyen y devuelven las respuestas HTTP correspondientes al cliente.

Esta capa actúa como el punto de entrada al sistema para todas las operaciones relacionadas con los usuarios.

- **Service:**

Esta capa contiene la lógica de negocio del sistema. Su responsabilidad principal es implementar las reglas y procesos que definen el comportamiento de la aplicación.

Los servicios reciben las solicitudes provenientes de los *controllers* y coordinan las operaciones necesarias para cumplirlas. Esto puede incluir validaciones adicionales, generación de tokens de sesión, procesamiento de datos y comunicación con la capa de persistencia.

Esta capa permite mantener separada la lógica de negocio del manejo de las solicitudes HTTP y del acceso directo a la base de datos.

- **Repository**

Esta capa se encarga de la comunicación con la base de datos. Su función es abstraer las operaciones de persistencia y proporcionar métodos para consultar, almacenar y actualizar la información del sistema.

Los *repositories* permiten acceder a las entidades almacenadas en la base de datos mediante consultas definidas a través del *framework* de persistencia utilizado. De esta manera, el resto del sistema puede interactuar con los datos sin depender directamente de los detalles de la base de datos.

Esta separación facilita la mantenibilidad del sistema y permite modificar o extender la lógica de persistencia sin afectar las demás capas de la aplicación.

En el flujo de lo implementado en nuestro sistema, **la arquitectura se ve aplicada** de la siguiente manera:

1. El cliente envía una solicitud HTTP con sus credenciales (email y contraseña).
2. El **Controller** recibe la solicitud y delega el procesamiento.
3. El **Service** valida las credenciales y gestiona la lógica de autenticación.
4. El **Repository** consulta la base de datos para verificar la información del usuario.
5. Si las credenciales son correctas, se genera un token y se guarda en la base de datos.
6. El sistema devuelve el token al cliente como respuesta.

Integración Frontend — Backend

A partir de la Iteración 2, el sistema cuenta con una interfaz de usuario que se comunica directamente con la API REST desarrollada en la Iteración 1. Ambos componentes operan de forma independiente y se conectan exclusivamente a través de peticiones HTTP.

El **frontend** es una aplicación web desarrollada en React. Su responsabilidad es presentar las vistas al usuario, capturar sus acciones y comunicarse con el backend para obtener o modificar información. No contiene lógica de negocio (reglas que definen cómo funciona el sistema): toda esa responsabilidad recae en el backend.

El **backend** es la API REST organizada en capas (Controller, Service, Repository) desarrollada en Spring Boot con Kotlin, tal como se describe en la sección anterior. Es el único componente que interactúa directamente con la base de datos.

La **comunicación** entre ambos componentes se realiza mediante peticiones HTTP estándar. El frontend consume los endpoints (puntos de acceso de la API) expuestos por el backend, enviando y recibiendo datos en formato JSON (un formato de texto estructurado para intercambiar información). Cuando una operación requiere autenticación, el frontend incluye el token de sesión en el encabezado Authorization de cada petición.

El flujo general del sistema integrado funciona de la siguiente manera:

1. El usuario interactúa con la interfaz visual en el navegador.

2. El frontend construye una petición HTTP y la envía al backend.
3. El backend procesa la solicitud a través de sus capas (Controller → Service → Repository).
4. El backend devuelve una respuesta en formato JSON.
5. El frontend interpreta la respuesta y actualiza la interfaz del usuario.

Integración de APIs TheDogAPI y TheCatAPI

El flujo de la interacción con los servicios de **TheDogAPI** y **TheCatAPI** es el siguiente:

1. **Comunicación con el Backend:** El cliente de servidor (`ExternalPetApiClient.kt`) realiza las peticiones HTTP externas a las APIs cuando localmente accedemos al endpoint `/api/external/breeds` .
2. **Comunicación en el Frontend:** El archivo principal de servicios de React (`api.js`) encapsula las llamadas HTTP locales con la función `getExternalBreeds(species)` .
3. **Optimización de Llamadas a las APIs:** Para evitar el *overfetching*, las respuestas de las APIs externas son mapeadas en el backend a objetos de transferencia de datos locales (`BreedDto.kt`). Así el frontend recibe exclusivamente una estructura limpia para mostrar la información en la interfaz de usuario. También primero buscamos en la base de datos local si ya se tiene registrada una raza antes de pedir la información detallada a una de las APIs.

8. Flujo de Autenticación

El flujo de autenticación del sistema funciona de la siguiente manera:

Cuando el usuario desea iniciar sesión, ingresa sus credenciales (correo electrónico y contraseña) en el formulario de login del frontend. Estas credenciales son enviadas al backend, que las valida contra la base de datos y, si son correctas, responde con un token de sesión (una cadena de texto única generada por el servidor para identificar al usuario durante esa sesión).

Una vez recibido, el token se guarda en el `sessionStorage` del navegador (un espacio de almacenamiento temporal que solo existe mientras la pestaña del navegador está abierta). Este token actúa como una credencial de identidad: cada vez que el usuario realiza una acción que requiere autenticación, el token es enviado automáticamente junto con la petición al backend en el encabezado `Authorization` (campo del mensaje HTTP que transporta información de identidad).

Las vistas que requieren autenticación están protegidas mediante rutas privadas (componentes que verifican la existencia del token antes de mostrar el contenido). Si el usuario intenta acceder a ellas sin tener un token activo en su sesión, el sistema lo redirige automáticamente a la pantalla de login.

Actualmente las vistas protegidas son:

- */home*
- */profile*
- */profile/edit*
- */my-posts*
- */favorites*
- */animal/:id*

Las vistas públicas (accesibles sin sesión activa) son */login*, */register*, */forgot-password* y */reset-password*.

Si el token no existe o ha sido eliminado (por ejemplo, al cerrar sesión o cerrar el navegador), el sistema considera que el usuario no está autenticado y le impide el acceso a cualquier funcionalidad que lo requiera. Se redirige a la página */login*.

9. Modelo Conceptual de Datos

El modelo de datos del sistema está conformado por las entidades Usuario, Animal, Raza (Breed), Foto (Photo) y Favorito (Animal Favorite), que en conjunto permiten gestionar usuarios, publicaciones de animales en adopción y el interés de los usuarios sobre ellos.

Entidad Usuario

Describe a los usuarios que interactúan con el sistema: pueden publicar animales en adopción, ver publicaciones de otros y manifestar interés.

Atributos

Atributo	Propósito
id_user	Identificador único del usuario. Generado automáticamente por la base de datos.
username	Nombre público del usuario.
first_name	Nombre de pila del usuario. Requerido, máximo 20 caracteres.
last_name	Apellido(s) del usuario. Requerido, máximo 50 caracteres.
email	Correo electrónico del usuario. Se utiliza como identificador para el inicio de sesión.
user_password	Contraseña del usuario. Se almacena como hash, nunca en texto plano.
zipcode	Código postal del usuario. Representa su ubicación aproximada. Debe tener entre 5 y 10 caracteres.
birth_date	Fecha de nacimiento del usuario. No puede ser una fecha futura.
gender	Género del usuario.
register_date	Fecha y hora en que el usuario se registró. Generada automáticamente.
token	Token de sesión activa. Se genera al iniciar sesión y se elimina al cerrar sesión.
reset_token	Token temporal de un solo uso para el restablecimiento de contraseña. Se genera al solicitar la recuperación y se invalida tras usarse.
reset_token_expiry	Fecha y hora de expiración del token de restablecimiento (vigencia de 1 hora).

Entidad Animal

Representa una mascota publicada para adopción. Cada animal está vinculado al usuario que lo publicó (publisher) y, opcionalmente, a una raza.

Atributo	Propósito
id_animal	Identificador único del animal. Generado automáticamente.

Atributo	Propósito
id_user	Referencia al usuario que publicó el animal.
id_breed	Referencia a la raza del animal. Es opcional (un animal puede no tener raza registrada).
animal_name	Nombre del animal. Requerido.
species	Especie del animal (DOG o CAT).
date_of_birth	Fecha de nacimiento estimada. La edad se calcula a partir de este dato, no se almacena.
description	Descripción del animal (temperamento, historia, etc.).
size	Tamaño del animal (small , medium , large , extra_large).
animal_zipcode	Código postal donde se ubica el animal. Entre 5 y 10 caracteres.
published_at	Fecha y hora de publicación. Generada automáticamente.

Entidad Raza (Breed)

Almacena el catálogo de razas conocidas para perros y gatos. Las razas se obtienen de una API externa y se guardan localmente la primera vez que se usan.

Atributo	Propósito
id_breed	Identificador único de la raza.
breed_name	Nombre de la raza.
origin	País o región de origen de la raza.
temperament	Descripción del comportamiento de la raza.
life_span	Esperanza de vida promedio (texto, ej. "10-14 years").

Entidad Foto (Photo)

Almacena las imágenes asociadas a un animal. Un animal puede tener varias fotos (relación 1:M). Las imágenes se alojan en Cloudinary y aquí solo se guarda su URL.

Atributo	Propósito
id_photo	Identificador único de la foto.

Atributo	Propósito
id_animal	Referencia al animal al que pertenece la foto.
url	URL pública de la imagen en Cloudinary.
width	Ancho de la imagen en píxeles.
height	Alto de la imagen en píxeles.

Entidad Favorito (Animal Favorite)

Tabla intermedia que registra el interés de un usuario sobre un animal (relación M:M). Su clave primaria compuesta evita intereses duplicados.

Atributo	Propósito
id_user	Referencia al usuario que manifestó interés.
id_animal	Referencia al animal de interés.
saved_at	Fecha y hora en que se registró el interés.

Consideraciones de seguridad

- La contraseña nunca se almacena en texto plano. Antes de guardarse, se le aplica el algoritmo de hash SHA-256, de forma que no pueda recuperarse su valor original.
- El token de sesión se genera como un UUID aleatorio al momento del inicio de sesión, lo que lo hace prácticamente imposible de predecir.
- El token de restablecimiento de contraseña también es un UUID aleatorio, de un solo uso y con vigencia limitada a 1 hora.
- La contraseña es excluida de las respuestas que el sistema envía al cliente, o en su defecto es reemplazada por asteriscos.
- Las credenciales de conexión a la base de datos se gestionan mediante variables de entorno, evitando que queden expuestas en el código fuente.

10. Limitaciones actuales

Las siguientes limitaciones han sido identificadas en el estado actual del sistema y serán resueltas en iteraciones futuras:

Campos birth_date y gender sin capturar en el registro

El formulario de registro no solicita fecha de nacimiento ni género al usuario. Al persistir el registro, estos campos se llenan con valores por defecto temporales.

Hash de contraseña sin salt

El algoritmo SHA-256 utilizado actualmente es vulnerable a ataques de diccionario (intentos masivos de contraseñas comunes). Se recomienda migrar a un algoritmo más seguro como bcrypt en iteraciones futuras.

Protección de rutas solo en el cliente

La protección de rutas se aplica únicamente en el frontend verificando si existe un token en sessionStorage (almacenamiento temporal del navegador). Un token manipulado manualmente podría intentar acceder a recursos del backend sin ser bloqueado de forma centralizada.

Sin persistencia de sesión entre pestañas

El token se almacena en sessionStorage, lo que significa que si el usuario abre una nueva pestaña del navegador deberá iniciar sesión nuevamente, ya que sessionStorage no se comparte entre pestañas.

Notificación por correo dependiente de configuración manual del servidor

El envío de correos al dueño del animal al manifestar interés requiere que el servidor tenga configuradas variables de entorno con una cuenta Gmail y su contraseña de aplicación. Si estas variables no están presentes, el interés se registra correctamente en la base de datos pero el correo no se envía, sin que el usuario reciba ningún aviso de fallo.

Sin validación de autointerés en adopción

El sistema no impide que un usuario marque "Me interesa" en un animal que él mismo registró. En ese caso, el correo de notificación se enviaría al mismo usuario que presionó el botón, lo cual carece de utilidad. Se recomienda agregar una validación en el backend que rechace la solicitud si el usuario autenticado es el mismo dueño del animal.

Estado del botón "Me interesa" sin persistencia entre sesiones

Al presionar el botón "Me interesa", este cambia visualmente a "¡Interés enviado!" únicamente durante la sesión activa en el navegador. Si el usuario recarga la página o cierra y vuelve a abrir el sitio, el botón aparecerá nuevamente disponible aunque el interés ya haya sido registrado. El backend rechazará correctamente el duplicado, pero la experiencia de usuario resulta inconsistente.

Nota de diseño: Unificación de Animal y Post

En versiones anteriores de este documento, el sistema distinguía entre dos entidades separadas: **Animal** (con sus datos biológicos) y **Post** (la publicación visible en el feed). A partir de la versión 4.0, ambas entidades fueron unificadas en una sola: el registro de un animal implica automáticamente su publicación en la plataforma.

Como consecuencia de este cambio:

- Ya no existe una entidad Post independiente.
- Crear un animal equivale a publicarlo.
- Editar o eliminar un animal equivale a editar o eliminar su entrada en el feed.
- Los casos de uso CU10 y CU13, que anteriormente referenciaban "publicación", ahora operan directamente sobre la entidad Animal.